Image Processing & Recognition: Lecture 3 Review

Welcome to Level 3! In previous lectures, we treated images as flat lists of numbers. Today, we treat images as **images**—2D grids of pixels with spatial relationships.

Part 1: What are you learning? (The "Human" Summary)

1. The Problem with Regular Neural Networks (MLPs)

If you try to feed a 1-megapixel image (1000×1000 pixels) into a standard MLP (from Lecture 2):

- **Parameter Explosion:** Just the first layer would need billions of weights.
- **Loss of Space:** Flattening an image destroys the information that "pixel A is next to pixel B."
- **No Translation Invariance:** If a cat moves from the left corner to the right corner, an MLP has to relearn what a cat looks like from scratch.

2. The Solution: Convolutional Neural Networks (CNNs)

Instead of connecting every neuron to every pixel, we use a small **Filter (or Kernel)**—like a 3×3 window—and slide it across the image.

- **Locality:** The filter only looks at small regions at a time.
- **Parameter Sharing:** We use the *same* filter for the whole image. If it learns to detect a vertical edge on the left, it can detect it on the right too.

3. Key Mechanics

- **Channels:** Images aren't just 2D; they are 3D volumes (Height $\times$ Width $\times$ Color Channels). A kernel must match the input depth (e.g., 3 for RGB).
- **Padding:** Adding zeros around the border so the image doesn't shrink after every layer.
- **Stride:** How many pixels the filter moves per step. Larger stride = smaller output map (downsampling).
- **Pooling:** A separate layer that shrinks the image by keeping only the max (Max Pooling) or average value in a window. This reduces computation and makes the model robust to small shifts.

4. Famous Architectures

- **LeNet:** The grandfather of CNNs (used for reading checks/digits).
- **AlexNet:** The deep network that started the AI boom in 2012 (used ReLU and GPUs).
- **VGG:** Made networks deeper by using small 3×3 blocks repeatedly.
- **ResNet:** Solved the problem of training *very* deep networks (100+ layers) using "Skip Connections" (adding the input to the output of a block).

Part 2: The Solved Problem (Step-by-Step)

Let's decode **Exercise 3** from your PDF (Page 116). This is a **Multi-Channel Convolution**.

The Task: Calculate the output of a convolutional layer and count the parameters.

- **Input (X):** 3×3 image with **3 channels** ($C_{in} = 3$).
- **Kernel (V):** 2×2 size ($k = 2$). It produces **2 output channels** ($C_{out} = 2$).
- **Bias (u):** 2 values (one for each output channel).

The Inputs (Matrices):

- **Input X (3 Channels):**
  - Ch 1: $\begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix}$
  - Ch 2: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$
  - Ch 3: $\begin{bmatrix} 2 & 3 & 4 \\ 5 & 6 & 7 \\ 8 & 9 & 10 \end{bmatrix}$
- **Kernels V (2 Filters, each has 3 channels):**
  - **Filter 1 (for Output 1):**
    - K1_Ch1: $\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}$
    - K1_Ch2: $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$
    - K1_Ch3: $\begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}$
    - Bias: 1
  - **Filter 2 (for Output 2):**
    - K2_Ch1: $\begin{bmatrix} 3 & 4 \\ 5 & 6 \end{bmatrix}$
    - K2_Ch2: $\begin{bmatrix} 4 & 5 \\ 6 & 7 \end{bmatrix}$
    - K2_Ch3: $\begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$
    - Bias: 2

Step 1: Calculate Output Dimensions

Formula: $n_{out} = n_{in} - k + 1$ (assuming stride 1, no padding). $3 - 2 + 1 = 2$. So, the output will be a 2×2 grid with 2 channels.

Step 2: Calculate Output Channel 1 (H_1)

To get the first output channel, we convolve **Input Ch 1 with Filter 1 Ch 1**, **Input Ch 2 with Filter 1 Ch 2**, and **Input Ch 3 with Filter 1 Ch 3**, then sum them all up and add Bias 1.

Top-Left Pixel Calculation:

- **Ch 1:** $(0 \cdot 0 + 1 \cdot 1 + 3 \cdot 2 + 4 \cdot 3) = 0 + 1 + 6 + 12 = 19$
- **Ch 2:** $(1 \cdot 1 + 2 \cdot 2 + 4 \cdot 3 + 5 \cdot 4) = 1 + 4 + 12 + 20 = 37$
- **Ch 3:** $(2 \cdot 2 + 3 \cdot 3 + 5 \cdot 4 + 6 \cdot 5) = 4 + 9 + 20 + 30 = 63$
- **Sum:** $19 + 37 + 63 = 119$
- **Add Bias (1):** $119 + 1 = 120$

(The solution in the PDF repeats this for the Top-Right, Bottom-Left, and Bottom-Right positions).

Final Result for Output Channel 1:

$$H_1 = \begin{bmatrix} 120 & 150 \\ 210 & 240 \end{bmatrix}$$

Step 3: Calculate Output Channel 2 (H_2)

We repeat the process using **Filter 2** and **Bias 2**.

Top-Left Pixel Calculation:

- **Ch 1:** $(0 \cdot 3 + 1 \cdot 4 + 3 \cdot 5 + 4 \cdot 6) = 0 + 4 + 15 + 24 = 43$
- **Ch 2:** $(1 \cdot 4 + 2 \cdot 5 + 4 \cdot 6 + 5 \cdot 7) = 4 + 10 + 24 + 35 = 73$
- **Ch 3:** $(2 \cdot 5 + 3 \cdot 6 + 5 \cdot 7 + 6 \cdot 8) = 10 + 18 + 35 + 48 = 111$
- **Sum:** $43 + 73 + 111 = 227$
- **Add Bias (2):** $227 + 2 = 229$

Final Result for Output Channel 2:

$$H_2 = \begin{bmatrix} 229 & 295 \\ 427 & 493 \end{bmatrix}$$

Step 4: Parameter Counting

How many learnable numbers are in this layer?

- **Weights:** (Kernel Height $\times$ Kernel Width $\times$ Input Channels) $\times$ Output Channels
  - $(2 \times 2 \times 3) \times 2 = 12 \times 2 = 24$ weights.
- **Biases:** 1 bias per Output Channel = 2 biases.
- **Total:** $24 + 2 = 26$.

Part 3: Your Turn (Interactive Exercises)

Practice Problem 1: The "Lite" Convolution

Let's simplify the math so you can practice the mechanics without a calculator.

Setup:

- **Input:** 3×3 image, **2 Channels**.
  - Ch 1: All 1s ($\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$)
  - Ch 2: All 0s ($\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$)
- **Kernel:** 2×2, **1 Output Channel**.
  - Filter Ch 1: All 1s ($\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$)
  - Filter Ch 2: All 2s ($\begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}$)
- **Bias:** 5.

Task: Calculate the top-left pixel of the output.

Solving Explanation:

1. **Channel 1 Convolution:** Take the top-left 2×2 of Input Ch 1 (all 1s) and multiply by Filter Ch 1 (all 1s). $(1 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 + 1 \cdot 1) = 4$.
2. **Channel 2 Convolution:** Take the top-left 2×2 of Input Ch 2 (all 0s) and multiply by Filter Ch 2 (all 2s). $(0 \cdot 2 + 0 \cdot 2 + 0 \cdot 2 + 0 \cdot 2) = 0$.
3. **Sum & Bias:** $4(\text{from Ch 1}) + 0(\text{from Ch 2}) + 5(\text{Bias}) = 9$.

Answer: The top-left pixel is **9**. (In fact, all pixels in the output will be 9).

Practice Problem 2: Parameter Counting (Exam Style)

Setup: You are designing a VGG-style network. You have a convolutional layer with the following specs:

- **Input Volume:** 224×224 pixels with **3 channels** (RGB).
- **Kernel Size:** 3×3.
- **Number of Filters (Output Channels):** 64.
- **Stride:** 1, **Padding:** 1.

Task: Calculate the total number of trainable parameters (Weights + Biases).

Solving Explanation:

1. **Analyze one filter:** A single filter must match the depth of the input. Size = $3\ (\text{height}) \times 3\ (\text{width}) \times 3\ (\text{input channels}) = 27$ weights.
2. **Account for all filters:** We have 64 such filters. Total Weights = $27 \times 64 = 1,728$.
3. **Account for biases:** Each filter has 1 bias term. Total Biases = 64.
4. **Grand Total:** $1,728 + 64 = 1,792$ parameters.

(Note: The input spatial size 224×224 does not affect the number of parameters, only the output size! This is a common trick question).